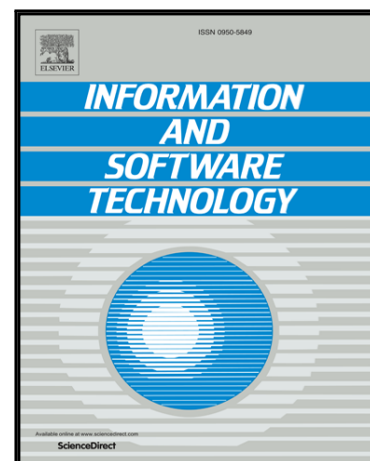


Journal Pre-proof

Requirements Specification for Developers in Agile Projects:
Evaluation by two Industrial Case Studies

Juliana Medeiros, Alexandre Vasconcelos, Carla Silva ,
Miguel Goulão

PII: S0950-5849(19)30201-0
DOI: <https://doi.org/10.1016/j.infsof.2019.106194>
Reference: INFSO 106194



To appear in: *Information and Software Technology*

Received date: 13 September 2018
Revised date: 17 August 2019
Accepted date: 20 September 2019

Please cite this article as: Juliana Medeiros, Alexandre Vasconcelos, Carla Silva , Miguel Goulão, Requirements Specification for Developers in Agile Projects: Evaluation by two Industrial Case Studies, *Information and Software Technology* (2019), doi: <https://doi.org/10.1016/j.infsof.2019.106194>

This is a PDF file of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability, but it is not yet the definitive version of record. This version will undergo additional copyediting, typesetting and review before it is published in its final form, but we are providing this version to give early visibility of the article. Please note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

© 2019 Published by Elsevier B.V.

Requirements Specification for Developers in Agile Projects: Evaluation by two Industrial Case Studies

Juliana Medeiros^{a,*}, Alexandre Vasconcelos^b, Carla Silva^b, Miguel Goulão^c

^a*Instituto Federal da Paraíba (UAI-IFPB), Brazil*

^b*Centro de Informática, Universidade Federal da Pernambuco (CIn-UFPE), Brazil*

^c*Faculdade de Ciências e Tecnologia, Universidade Nova de Lisboa, (NOVA LINCS), Portugal*

Abstract

Context: An inadequate requirements specification activity acts as a catalyst to other problems, such as low team productivity and difficulty in maintaining software. Although Agile Software Development (ASD) has grown in recent years, research pointed out several limitations concerning its requirements engineering activities, such as Software Requirements Specification (SRS) provided in high level and targeted to the customer, lack of information required to perform design activities and low availability of the customer. To overcome these issues, the RSD (Requirements Specification for Developers) approach was proposed to create an SRS that provides information closer to development needs. In addition, existing literature reviews identify a demand for more empirical studies on the requirements specification activity in ASD. **Objective:** Face to this, this work presents the evaluation of the RSD approach with respect to how it affects the teamwork and to identify its strengths and limitations. **Methods:** This evaluation was performed by means of two industrial case studies conducted using a multiple-case design, focusing on software engineers as the analysis unit. Data were collected during 15 months from documents, observations, and interviews. They were triangulated, analyzed, and synthesized using techniques of grounded theory. **Results:** The findings pointed out that the readability of SRS was compromised when several requirements are specified in the same RSD artifact. Evaluation also indicated the need of prioritization and categorization of the acceptance criteria, a tool for creating, searching and tracing the artifacts, and obtaining acceptance tests from acceptance criteria. On the other hand, the findings showed that the practices used to specify requirements using the RSD approach have the potential to produce a more objective SRS, tailored for the development team. **Conclusions:** As a consequence, the structure of the RSD artifact was considered as a factor that improved the team performance in the two case studies.

Keywords: Software Requirements Specification. Agile Software Development. Empirical Software Engineering.

1. Introduction

A survey conducted by VersionOne [1], involving about 4000 people, showed that 45% of respondents use agile methods in most projects. However, empirical studies conducted in the industry point out several problems related to Requirements Engineering in ASD [2, 3, 4, 5], such as low availability of the customer, poor quality of Software Requirements Specifications (SRSs), inadequate management and prioritization of the requirements, and dealing with Non-Functional Requirements (NFRs), among others. Moreover, recent literature reviews point out that the requirements specification activity is a challenge affecting ASD [6, 7, 8, 9, 10]. In general, SRSs in ASD are insufficient and inadequate to be used by the devel-

opment team, making the development harder at a later stage [2, 8, 10, 11, 12].

User Stories are widely adopted in ASD as SRSs. Yet, User Stories are too often poorly written in practice and exhibit inherent quality defects [13]. User Stories are written in the language of the problem domain and its format leads to a high level description of the software requirements, targeting the customer. User stories lack the expressiveness to describe design requirements [8]. As a consequence, developers consider that SRSs based on User Stories are brief, vague, ambiguous and insufficient for capturing the complexities of the up-front design [4, 14].

Only simple, customer visible functional requirements can be described as basic User Stories. When the software system is large and complex or hardware-dependent, User Stories do not convey enough information for software design, and separate system and subsystem requirements are required [8]. Story Cards are extensions of User Stories and are also used to specify requirements in agile projects. A Story Card is like an iceberg: what you see is only a small part of what you will get [15]. Sharp et al. [16] consider

*Corresponding author

Email addresses: juliana.medeiros@ifpb.edu.br (Juliana Medeiros), amlv@c.in.ufpe.br (Alexandre Vasconcelos), ctlls@c.in.ufpe.br (Carla Silva), mgoul@fct.unl.pt (Miguel Goulão)

that Story Cards have an incomplete and partial view of the requirements, which hampers the development from them.

Motivated by this scenario, we performed a systematic mapping [17] which pointed out several factors (positive and negative) related to the quality of software requirements specification in ASD. These factors have been confirmed by evidences obtained from six case studies we have conducted [18] that investigated the phenomenon of the requirements specification activity in different agile projects. These case studies resulted in an explanatory model providing a deep description of the factors that should be considered in order to write more useful SRSs in ASD. The findings of these investigations are also reinforced by other studies that indicate that SRSs in agile projects are insufficient for the coding activity [6, 8].

Given the challenges related to the requirements specification activity in ASD and the evidences obtained from the six case studies [18], we created the Requirements Specification for Developer (RSD) approach [19], aiming to support the requirements specification activity in ASD and overcome the aforementioned limitations.

RSD approach attempts to address some of the factors that directly affect the quality of SRSs in ASD, according to the model built from the cross-case synthesis, for example: Team-Oriented, Objectivity, Technical Aspects, Consolidated Information and NFRs [18]. The writing of a simple and clear SRS, without ambiguity and redundancies is considered a relevant quality factor. On the other hand, a prolix and fragmented description of the requirements compromises the quality of the SRS.

The model also establishes that the SRS in ASD should be directed to the development team, so it should be closer to what will be implemented. Therefore, RSD is designed for the developer, and it is not tailored to validate requirements with customers.

Further, the model shows that the quality of SRSs is also affected indirectly by: Stakeholders' factors such as inadequate experience of the team and low customer availability; Organizational factors such as using the SRS to validate requirements with customers and performing late software validations; and External factors such as contract agreements and characteristics related to the software domain [18]. These factors are related to the companies' context and affect the quality of SRSs, regardless of the approach used to specify the requirements (User Stories, Use Cases or RSD, for example). Thus, these factors are not addressed by RSD approach.

RSD provides an integrated view of the requirements by linking, in a systematic way, the benefits of identifying the problem domain concepts (conceptual modeling), the visual representation of interface requirements (mockups), the acceptance criteria composed of business rules, NFRs and technical constraints. Although these views are meant to capture requirements, they are very close to the field of implementation and can help in the software maintenance and knowledge transfer.

This paper presents the evaluation of the RSD approach through two case studies conducted in industrial agile projects. In fact, empirical studies conducted in the scope of the requirements specification activity have been developed for more than 20 years and have produced important knowledge. However, the vast majority of studies were not conducted in the context of ASD. According to Schön et al. [7], agile practices in Requirements Engineering need additional attention as a research theme, and more empirical studies are required to better understand the impact of agile practices in Requirements Engineering.

The goal of this empirical study was to evaluate the impact of the RSD in practice and identify its strengths and limitations. More specifically, we intend to answer the following Research Questions (RQs):

- RQ1: How does the team evaluate the SRS produced using the RSD approach?
- RQ2: How does the RSD approach affect the team work?

We did not have the intention of making a comparative assessment between RSD and other approaches, although a few questions were asked considering requirements specification techniques previously used by the interviewed practitioners.

The remainder of this paper is organized as follows: Section 2 presents an overview of the RSD approach. Section 3 describes the research method used. Section 4 describes the results of each case study. Section 5 discusses the results of the cross-case analysis and the threats to validity. Some related works are discussed in section 6. Finally, section 7 presents our conclusions and directions for future work.

2. RSD Approach Overview

2.1. Derivation of the RSD approach

The RSD approach [19] was built based on the findings from a systematic mapping study [17], the results from six case studies in industry [18] and the Agile Manifesto [20], which establishes using frequent software deliveries for validating requirements, rather than using the SRSs. Thus, SRSs should be tailored for the development team, rather than the customer. RSD approach can replace other techniques used in ASD, such as User Stories and Use Cases, but overcoming the limitations described previously. Thus, RSD can be used with XP, Scrum, or any other agile method where the client validates the requirements through working software, as established in the Agile Manifesto [20]. Based on this agile principle, RSD approach focus on the development team and the SRS produced is not intended to be used as a mechanism to requirements validation with the customer.

2.2. Design Practices and Structure of the RSD Approach

The RSD approach proposes the adoption of three well-established design practices that make the SRS targeted to the software engineer: Conceptual Modeling, Mockups Modeling and Specification of Acceptance Criteria+ (AC+, an extension of the Acceptance Criteria to be further explained in this section).

The findings from the investigations in the literature and industry also pointed out that frequent change requests are factors that compromise the productivity of the development team, because the software architecture usually presents scalability problems resulting from the negligence in the design activities. To reduce these problems, the RSD approach includes the conceptual modeling to identify the business concepts (entities, attributes, and relationships). This practice can contribute to build a more scalable architecture and can reduce rework required to meet the frequent changes.

Mockups modeling helps to identify the visual interface elements between the system and the user. The conceptual modeling and the use of mockups are consolidated practices in traditional development. Although these practices are not part of the main agile methods, they are used in many agile projects, even if they aren't used in a systematic way [21], nor integrated into the functional requirements outlined by user stories. RSD systematizes the use of these practices in ASD by integrating them into a single view aiming to provide an SRS with enough information for coding.

To have an SRS targeted for the developer, an AC+ defines not only business rules, but also validation rules, interface, technical or any other type of constraint necessary for the system coding. An AC+ can be classified into six types:

- Business - Represents a restriction related to intrinsic nature of the business;
- Validation - Represents a validation that the application needs to perform, but that is not directly related to the core business;
- Interface - Represents any restriction related to the user interface. A widget must be enabled (or disabled) depending on some condition or the content of another widget;
- Technical - Represents a technical restriction on how the solution should be implemented. Often, they are rules associated with the confirmation action buttons;
- Non-Functional - Represents concerns about tracking quality, e.g., performance constraints, reliability constraints;
- Other - When it does not fit in any of the previous types.

A requirement may have AC+s with different priorities that can also be allocated to different sprints. In addition, new AC+s can be identified in any sprint throughout the development process. Some examples of AC+s are presented in Table 1. These AC+s are related to the "Registration of athlete", a requirement of an information system for doping control for a federation affiliated to the International Association of Athletics Federations [22]. The Type column refers to one of the types of AC+ defined in previous paragraph. The last column defines whether the AC+ is of the General category (G) because it has the potential to be reused, or if it is Specific (S) to a requirement. AC+ initially classified as Specific may be reclassified as General when an opportunity for reuse is identified later. This categorization was suggested by the team that evaluated the RSD approach in the first case study. These improvements were incorporated into the RSD approach.

Table 1: Acceptance Criteria Examples

ID	Description	Type	Cat.
AC01	The email address must be valid.	V	G
AC03	To save, it is necessary that all required fields (*) are filled.	V	G
AC04	Only active records must be displayed.	V	G
AC07	The age must be calculated from the date of birth.	V	G
AC08	The routine to save an athlete should also save the corresponding addresses.	T	S
AC09	The operation to read and write files in the file system should be done through relative address.	T	G
AC12	The sequential code to identify the record must be generated by the database.	T	G
AC13	The initials of the athlete must be extracted from the athlete's name, e.g., if the name is "Fabiana de Almeida Murer", initials must be "F.A.M".	T	S
AC17	All foreign athletes must have a passport number.	B	S
AC20	The drop-down list must only display the confederations that the user logged has access permission in his/her profile.	B	G
AC21	There cannot be two athletes with the same registration number in the same confederation.	B	S
AC50	The label must use the multilingual resource.	N	G
AC90	The widget is read-only. It cannot be changed by the user.	I	G

The AC+s classified as being of the General Category and of the type Validation, Interface or Non-Functional have a potential to be reused in the same project or in other projects, as can be seen in the examples shown in the Table 1. However, how to store and specify an AC+ can contribute or not to facilitate its reuse. The approach does not impose the adoption of any particular tool. The AC+s can be stored in a kind of repository to encourage reuse during the specification and coding activities. A repository can be operationalized through a shared document among stakeholders or by using any tool to support the requirements activities or even the project management. The openproject tool¹, for example, provides a mechanism for storing and categorizing acceptance criteria.

¹<https://www.openproject.org/>

The structure of the SRS created with the RSD approach (from now on called RSD artifact) joins business concepts, mockups and AC+s offering an integrated vision for the developer. To illustrate its use, Figure 1 shows the RSD artifact related to the “Registration of athlete” requirement of the same doping control system. The RSD artifact is divided into five parts.

The first part (top) identifies the requirement by means of a label, high-level description, priority, requestor stakeholder, and sprint in which it will be implemented. The second part shows the mockups associated with the requirement, if applicable. Notice, in the example, that the use of the mockup allows the visualization of the data of the athletes and how they will be presented in the system, which facilitates the development team work. The third part (the left column in the table) presents widgets that are present in the mockup. The fourth part (center column) shows the data entities and attributes extracted from the conceptual model which are related to each widget. Finally, the fifth part (right column) shows the AC+s related to the widgets and data entities.

AC+s may be reused by different requirements. Reuse may also occur several times in the same RSD, for example, AC90. An AC+ does not have to be associated to a data entity (e.g. AC03) nor to a widget (e.g. AC12). In general, AC+s which describe NFRs, web services or algorithms have no relation with widgets. In this example, note that the AC+s can be used to detail business-related needs (AC17 and AC21), but also to describe information that is closer to what will be implemented, such as validation constraints (AC01), interface (AC90), technical (AC09) and NFR (AC50) (see Table 1). RSD allows all these features to be represented in a single and integrated form which may facilitate the developer’s understanding.

RSD provides an integrated view of the requirements, systematically linking the problem domain concepts (conceptual modeling), the visual representation of interface requirements (mockups), the business rules, NFRs and technical constraints (AC+s). While these views are meant to capture requirements, they are very close to the field of implementation, providing a wider requirements coverage to the development team when compared to User Stories, for example, which only address user requirements.

3. Evaluation Method

Case studies are powerful methods to be used when contemporary events cannot be controlled by the researcher [23], or in situations where the context is expected to play an important role, so that the reductionism of controlled experiments would be inappropriate, or where effects are expected to be wide ranged, or take a long time (e.g. weeks, months, years) to appear [24].

Therefore, we chose case study as the research method to evaluate the RSD approach because the real context in

practice is essential in the evaluation process and we would have to consider the following variables of the real-world:

- The whole ASD lifecycle which requires the customer collaboration as stated by the principles of the Agile Manifesto. In fact, collaboration with the customer is required to elicitate requirements, to clarify doubts during the coding activity and to validate partial versions of the software. Thus, the iteration with the customer is an important variable in ASD;
- The observation of the difficulties and facilities to code and test based on the RSD artifact. We cannot observe properly these activities evaluating the requirements in an isolated manner. Thus, it is necessary to investigate the entire development cycle in an integrated way;
- The reuse of requirements, dependence among stakeholders, rework, impact analysis of change requests, and knowledge transfer. These are variables that require the observation of the use of the approach in a real-world setting for some months and involving several sprints.

So, we cannot assess these variables easily through controlled experiments. Besides, according to Shields [25] (as cited in [26]):

"The strength of qualitative approaches is that they account for and include difference – ideologically, epistemological, methodological – and most importantly, humanly. They do not attempt to eliminate what cannot be discounted. They do not attempt to simplify what cannot be simplified."

Hence, we assumed the principle that the assessment of the RSD approach cannot be separated from the real context. Thus, we conducted two empirical qualitative case studies to assess how the RSD approach works in practice and gather insights to enhance it.

The case studies were conducted following the guidelines suggested by Runeson and Höst [27], composed of five steps: Design and Planning (section 3.1); Preparation (section 3.2); Collecting evidence (section 3.3); Analyzing the data collected (section 3.4); and Synthesis (section 4).

3.1. Design and Planning

In this step, the objectives and the case studies protocol were defined. The goal of the case studies was to assess how the RSD approach works in practice. The studies had an explanatory purpose [27] following a single standard protocol that describes the procedures to collect and analyze the data. An interview script was designed to guide the interviews with software engineers (See section 3.3.3).

Yin [23] distinguishes between holistic case studies, where the case is studied as a whole and has only one

Label: Registration of athlete **Priority:** Critical **Source:** Ana **Sprint:** 1
Description: The system should enable the inclusion and updating of data of national and foreign athletes of sports federations recognized by the International Olympic Committee

Athlete _ _ X



53 x 46

Photo

Full Name Last Name Initials

Birth Date Age Gender Passport Number

☐ Foreign

Email Phone

Confederation Register Number

Widget	Concepts	Acceptance Criteria
Photo	person.name	AC09
Full Name	person.fullName	-
Last Name	person.lastName	-
Initials	athlete.initialsName	AC13, AC90
Birth Date	athlete.birthDate	AC05, AC06
Age	-	AC07, AC90
Gender	person.gender	-
Foreign	person.isForeign	AC17
Passport Number	person.passportNumber	AC17
Email	person.email	AC17
Phone	person.phone	-
Confederation	confederation.name	AC04, AC20
Register Number	athlete.registerNumber	AC21
Save	-	AC03, AC08
-	person.idPerson	AC12

Widgets of the Mockup
Data Entities
AC+

Figure 1: Structure of RSD artifact [19]

unit of analysis, and embedded case studies in which multiple units of analysis are studied within a case. The case studies presented in this work are classified as two separate holistic cases because each one of them has just one unit of analysis (software engineer).

The research questions previously presented in Section 1 are detailed in the following:

- RQ1: How does the team evaluate the SRS produced using the RSD approach?
 - The development team evaluates the suitability of the RSD approach for specifying requirements in agile projects. Thus, the evaluation is focused on two perspectives: content and structure (type and organization of information) of the SRSs created using the RSD approach.
- RQ2: How does the RSD approach affect the team work?
 - The model created in [18] is used as a reference to guide the analysis on how the RSD approach affects the team work. The development team evaluated the effort required to specify, code and test, as well as, the impact of the RSD in software non-conformities, impact analysis of

change requests, reuse of SRSs, knowledge transfer, customer collaboration and dependence between stakeholders.

3.1.1. The Sample

In the context of this research, agile projects are those guided by the values and principles of the agile manifesto [20]. The precondition for selection of the companies was the existence of projects that used agile practices, such as Frequent Delivery, Continuous Improvement and Self-Organizing Teams. According to Ashmore and Runyan [28] agile practices like these are considered determinants to characterize an ASD. To conduct the investigation, we were conditioned on acceptance and willingness of companies to use the RSD approach in one of their projects.

The first case study was conducted over 12 months in the development of an information system for doping control [22] for a federation affiliated to the International Association of Athletics Federations. The project covered features from the registration of the athlete to the judgment, when the test result is positive. The development team consisted of ten software engineers (two system analysts and eight developers).

The second study was conducted over 3 months in a small company established in 2004. The company has cho-

sen one of its three development teams that had four software engineers (all developers) responsible for the evolution and maintenance of three information systems. The first system is for the management of residential condominiums and it is running on more than 40 customers. The second is for a financial cooperative, and the third system is for the company's internal administrative control.

Both companies used Scrum and XP practices. The profile of these companies and of the software engineers are summarized in Table 2 and in Table 4 (Section 3.3.3), respectively.

Table 2: Context of the two case studies

	Case Study 1	Case Study 2
Company Size	Small	Small
Period Investigation	12 months	3 months
Number of Projects	1	3
Type of Software	Information Systems	Information Systems
Project stage when the approach was introduced	The project was still in the planning stage	Projects already in progress and working on more than 40 customers
Development Team	10 software engineers (2 trainees)	4 software engineers (2 trainees)
Roles	System Analysts (2) and Developers (8)	There was no division of roles
Agile practices used	Backlog, frequent releases, iterations, version control, continuous integration, automated build, refactoring	Backlog, planning poker, frequent releases, continuous integration, iterations, version control, retrospective, daily meetings
RSD approach practices	Use of the three practices equally (Modeling Concepts, Mockups and Specify AC+)	Focus on the specification of AC+
Tools for Req. Specific.	MS Word	Redmine
Sprint Duration	Monthly	Monthly
Internal Releases	Weekly	Monthly
Acceptance Tests	Weekly	Not frequent

3.2. Preparation

Before the initiation of a case study in each company, there was a conversation with the team about its expectations on adopting a new approach to replace user stories. In the final stage of preparation, training was conducted in each company to introduce the RSD approach to the teams. The practices used in the RSD approach were detailed for the software engineers. Then, a pilot was performed with some requirements that were specified using the RSD approach and then implemented. The pilot aimed to clarify doubts, and familiarize the team with the new approach.

3.3. Collecting evidence

In each case study, data was collected and analyzed simultaneously by one researcher (the first author), in incremental and iterative steps to answer the research questions. Besides, another researcher (a master student not

authoring this paper) reviewed the collected and analyzed data. Table 3 shows the data collected by source.

Table 3: Data Collected by source

Data	Observations	Documents	Interviews
Team Evaluation	The content and structure of the RSDs	X	X
	The effort required to specify requirements, code and test using RSD approach	X	X
	The difficulties that software engineers had when using the RSD approach.	X	X
	Dependence between stakeholders	X	X
	Impact analysis of change requests	X	X
Amount of changes made in the RSD (volatility)		X	X
Type and number of Non-Conformities (NC) identified in the software by the team during acceptance tests		X	X

3.3.1. Observations

The researchers (the first author and the same master student mentioned before) did not work on the activities of the development process. They just participated as observers in some activities carried out by the team. Hence, they neither interfered in the way the projects were being conducted, nor made any suggestions or criticisms.

Observations were made within the companies during the period of each study. At least twice a month, the researchers participated in some meetings acting as observers, taking notes, perceiving how software engineers were using the RSD approach and the difficulties faced. For example, they participated during 3/4 hours in activities such as: (i) Discovery sessions with the customer to detail their needs, and to specify the AC+s, mockups, and the conceptual model; (ii) Team meetings to share better practices, problems, and difficulties that compromised the specification, coding and test activities; (iii) Meetings involving the development team, the product owner and the customer to perform acceptance tests; (iv) Meetings to discuss and analyze the impact of change requests to requirements; (v) Sprints Planning.

3.3.2. Documents

The RSD artifacts and test records produced by the development teams helped to understand the difficulties pointed out by the software engineers during the meetings and interviews. In the first case study, 39 RSD artifacts were elaborated by the systems analyst in collaboration with the product owner and, sometimes, with the customer. In the second case study, 26 RSD artifacts were elaborated by the internal product owner in collaboration with the team.

Each RSD artifact was evaluated by the software engineer in charge of coding it. He/she evaluated each RSD artifact according to six quality factors of the model obtained from the cross-case analysis of six industrial case studies [18]: Team-Oriented, Objectivity, Readability, Clearness, Completeness and Outdated. Each factor was assessed as in Compliance or Non-Compliance. The results of this

evaluation are shown later in Section 4. In addition to collecting data from the RSD artifacts, the researchers also collected data from the evaluation made by the software engineers in the RSD artifacts.

3.3.3. Interviews

Interviews were conducted individually with each software engineer, described in Table 4, at the end of each case study. Each interview lasted from 30 to 60 minutes. Before each interview, it was explained that the purpose of the interview was to evaluate the approach and propose improvements to it. Each interviewee was made aware of the importance to detail the answers as much as possible. The majority of the questions were open-ended, and only a few were closed. The questions were organized in a pre-established order by subject. However, if the interviewee mentioned a subject out of order, we changed the question order, aiming not to interrupt the reasoning of the respondent. The interview script used to guide the interviews has 34 questions (Q) and it is available at <https://sites.google.com/site/rsdapproach/interviewguide>. It addresses the following aspects:

- Experience of the Interviewee (Q1 to Q5);
- The process of learning the RSD approach (Q6 to Q8);
- Content and structure of the RSD artifacts (Q9 and Q14);
- Effects of the RSD approach on the work of the team (Q15 to Q17);
- Dependences between stakeholders (Q18 to Q19);
- Effort required to specify, code and test using the RSD approach (Q20 to Q27);
- Impact analysis of change requests (Q28 to Q30);
- Opportunities for improving the RSD approach (Q31 to Q34).

Besides registering answers, we also recorded the observations of the interviewees that were out of the original questions list but were useful for the study. At the end of each interview, answers were reviewed with the interviewee to confirm if they had been correctly noted, and to capture complementary information, if needed. The profile of the software engineers interviewed is described in Table 4, which indicates the ID, Gender, Degree, Role and Experience of the participants of each case study. CS1 identifies the participants of Case Study 1 and CS2 identifies the participants of Case Study 2.

3.4. Analyzing the data collected

As previously described, the content of each RSD artifact was evaluated only by the software engineer responsible for coding it. In contrast, the RSD artifact structure was evaluated by the entire team. During the interviews, the RSD artifact structure was evaluated according

Table 4: Profile of the software engineers interviewed in this study

ID	Gender	Degree	Role	Experience
#1CS1	Male	Bachelor	Developer	4 year
#2CS1	Male	Bachelor	Developer	5 years
#3CS1	Female	Bachelor	System Analyst (SA)	5 years
#4CS1	Male	Specialist	Developer	5 years
#5CS1	Male	Bachelor	Developer	4 years
#6CS1	Male	Doctoral	SA	10 years
#7CS1	Female	Master	Developer	8 years
#8CS1	Male	Bachelor	Developer	6 years
#9CS1	Male	Student	Developer	0.5 year
#10CS1	Male	Student	Developer	0.5 year
#1CS2	Male	Bachelor	Developer/SA	5 years
#2CS2	Male	Bachelor	Developer/SA	3 years
#3CS2	Male	Student	Developer/SA	0.5 year
#4CS2	Male	Student	Developer/SA	0.5 year

to other quality factors of the model previously mentioned [18]: Simplicity, Acceptance Criteria, Non-Functional Requirements, Technical Aspects, Functional Requirements, Consolidated Information, Automated Support, Traceability and Change History. Each factor was assessed as being in Compliance, Partial Compliance or Non-Compliance. The results of this assessment are shown later in Section 4.

The interviews were transcribed, and then techniques of grounded theory were used to code, categorize and synthesize the data in each case study, as proposed in the constant comparison method [29]. The coding process involves attaching codes, or labels, to pieces of text that are relevant to a particular theme or idea of interest in the study. After coding the transcriptions, the codes were reviewed to identify similarities, duplicates, or misleading codes. The codes discovered from the transcription of each interview were constantly compared to codes in the same interview and from other interviews. From the constant comparisons of the codes, similar codes were fused to become a single factor that affect the team work using the RSD approach (as described in Section 5). Figure 2 shows an example of the codes building process.

The data collected in the interviews were triangulated with the data obtained from the documents and observations in order to increase the credibility of the data. According to Merriam [26], internal validity assesses the degree to which the research findings match reality and triangulation is a strategy to promote findings validity and reliability by using multiple investigators, data sources and collection methods. Triangulation prevents the influence of individual analysis based on interviewer-researcher's personal opinion [30].

4. Results

In this section, for each case study, the evaluation results are organized by research questions. Also, the context of each case study is described. A summary of the contexts was shown in Table 2 (Section 3.1.1).

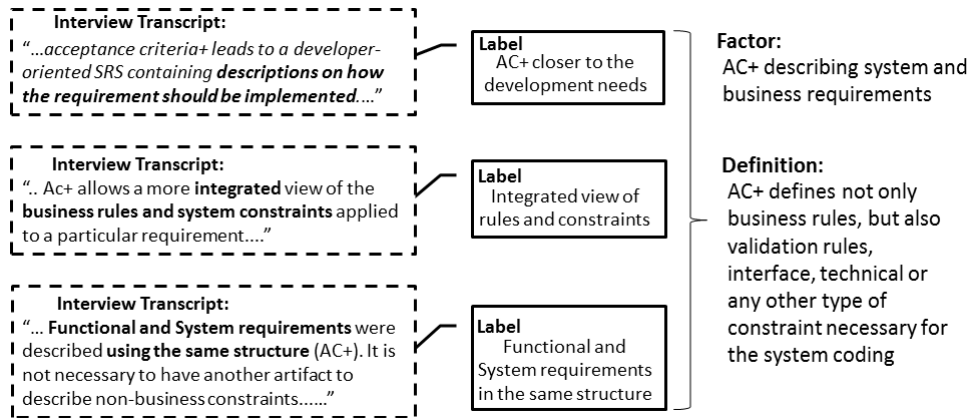


Figure 2: Factors building example

4.1. Case Study 1

4.1.1. Context

The RSD approach was adopted since the beginning of the project. Each sprint lasted for a month, but partial versions of the software were released every week for the internal acceptance testing. Several Scrum and XP practices were well established in the project, such as backlog, iterations, frequent releases, version control, continuous integration, refactoring, automated build and retrospective [31]. Daily meetings were conducted to monitor the project status, to share better practices, and to discuss the problems and difficulties faced.

The project used different people to play the roles of analyst and developer. The development team was composed of 10 software engineers: 8 developers and 2 system analysts. One of the analysts and six developers worked together on other projects. However, one of the analysts and two trainees (developers) were novice in the team. Except for the two trainees, all other team members had more than four years of experience with ASD. All developers have already played the role of system analysts in other projects, as well as both analysts have already worked as developers. The most experienced developer played the roles of architect and configuration engineer, and was responsible for conducting deployments weekly.

The analysts elaborated the RSD in collaboration with the internal product owner and the customer during the weekly discovery sessions. The Microsoft Word was used to elaborate the RSD. The Java language was used to code the requirements. The product backlog was controlled through a Microsoft Excel spreadsheet. Redmine tool was used for the project management and to report the results of acceptance tests. The analysts used Pencil tool for modeling the mockups and Astah tool for the conceptual modeling. The evaluation results are presented as follows by research questions.

4.1.2. RQ1: How does the team evaluate the SRS produced using the RSD approach?

They evaluated the RSD approach from two aspects: content and structure (type and organization of information).

a) *Content*. The developers evaluated all 39 RSD artifacts and 257 AC+s produced during the case study. Some RSD artifacts specified more than one requirement. All RSD artifacts were evaluated as in compliance with Team-oriented and Updated (represented in the model as Outdated) factors.

Eleven RSD artifacts (28%) presented non-compliance with at least one quality factor. Most of the RSD artifacts (72%) were evaluated in compliance with all factors. According to the developers, 95% of RSD artifacts (37) were straightforward. Two RSD artifacts were considered non-objective because they had redundant information, unnecessary for the developer. Despite this, the acceptance tests did not present any Non-Conformities (NC) in the functionalities related to these two RSD artifacts. According to the interviews (Q22 in interview script), the analysts' experience contributed to write the AC+s in an objective way for the developer.

The team considered that 36 (92%) of the RSD artifacts were easy to understand. The Readability factor was considered compromised in three RSD artifacts that described more than one requirement in the same document. For example, the requirements "Solicitation of a Doping Kit", "Search of Solicitations", and "Approve a Solicitation" were described in the same RSD artifact. In the opinion of the developers (Q32), each requirement of the product backlog should be specified in a separate RSD artifact to increase readability.

The requirements were described clearly (without ambiguity) in 95% (37) of the RSD artifacts. Two RSD artifacts were considered non-compliant with the Clearness factor. One of the non-compliant artifacts did not clearly define how the data access restriction should be done. In the other, it was not clear whether the data should be persisted in the database or not. Two RSD artifacts considered am-

biguous were also evaluated as non-compliant in relation to the Readability factor, because they had more than one requirement described in the same document. Regarding the 257 AC+s specified during the period of the case study, only 6% (15) presented problems of lack of clarity.

Regarding the Completeness factor, 85% (33) of the RSD artifacts were considered sufficient for the developer to code, without the need to consult complementary sources. Six RSD artifacts were evaluated as incomplete due to the lack of information needed to code some features. Two RSD artifacts that had prolix descriptions unnecessary for the developer were also evaluated as incomplete. Thus, the descriptions with excessive detail did not contribute to make a complete documentation. Although the acceptance tests related to these RSD artifacts did not find NC in the software, the lack of objectivity and completeness affected the productivity of the team given that the developers needed to talk with other stakeholders or to look for other sources to clarify related doubts, as reported by one developer (Q21):

"Sometimes we identify that the SRS is incomplete, lacks some AC+. So, we lose a lot of time because we need to get the information with the analyst or wait for the documentation to be updated." (#9CS1)

In the interviews, we inquired the ten software engineers about the content of the RSD artifacts compared to the content of the SRSs used in other projects (Q13). User stories and use cases were cited by more than 70% of respondents as techniques that they had used previously. Regarding the Completeness factor, nine software engineers considered the RSD approach as better, while just one considered it as worse than the previously used approaches. Regarding the Clearness factor, half of the respondents considered that the RSD approach is better than other approaches they have used. The other half considered that the RSD artifact has the same quality. All respondents pointed out that the objectivity and readability of the RSD artifact is better than the SRS artifact created when using other approaches.

Finally, we analyzed the changes (volatility) made in the RSD artifacts to meet the stakeholders' needs. A total of 72 changes were identified. Some RSD artifacts were changed more than once, but a large number of changes in a RSD artifact did not mean it became more complete, nor the software had few NC in the acceptance tests. For example, the RSD artifact #5 had many changes, but it still has presented a lot of NC in the software. Although some changes were made in the RSD artifact #29, they were not enough, since the developer has assessed it as incomplete. In such requirements, it is possible that the team did not have the necessary understanding of customer needs. Although many changes have been made in the RSD artifacts related to these requirements, they were not considered complete and produced a lot of NC.

Thirteen RSD artifacts (33%) did not have their content changed.

b) Structure.

The software engineers also were interviewed about how they assessed the RSD artifact structure based on mock-ups, concepts and AC+. Q9 was an open section regarding this subject. In Q10, the opinion of the software engineers should be according to a scale from 1-Inadequate to 5-Very Adequate. In general, the RSD artifact structure was Very Adequate in the opinion of most respondents (8). The remaining respondents (2) evaluated it as Adequate. Then, the team was inquired about each quality factors (Q11). All ten software engineers pointed out that the structure was in compliance with the Simplicity, Acceptance Criteria, Technical Aspects and Functional Requirements factors, as summarized in Figure 3.

One system analyst evaluated the Non-Functional Requirements factor as in Partial Compliance (Q11). He suggested (Q31) the definition of an initial catalog of the NFR to improve the adoption of the RSD approach, as follows:

"Although the structure predicts the Non-Functional requirements type to describe acceptance criteria, in practice, the team neglected the specification of these requirements. The project repository had few acceptance criteria of this type. The approach could suggest a catalog with generic NFR to be part of the initial repository of each project." (#4CS1)

The RSD approach does not impose the adoption of a particular tool for the specification of AC+, construction of mockups and the conceptual model. It is up to each company to use the tools according to their needs, processes, and methodologies. The development team didn't report any kind of problem regarding the use of the Pencil tool for the elaboration of mockups and the use of Astah tool for the conceptual modeling (Q17). However, the team reported difficulties regarding the use of Microsoft Word to store and control the repository of AC+s and to elaborate the RSD artifacts.

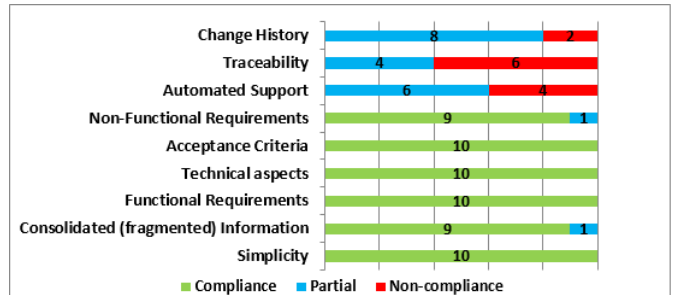


Figure 3: Evaluation of the RSD artifact structure by quality factor – case study 1

In the opinion of system analysts (Q17), the operationalization of the repository through a text document does

not facilitate the search for AC+s to be reused. Moreover, traceability has been undermined because the relationship between requirements and acceptance criteria was made manually using an MS Excel worksheet instead of being generated automatically. Thus, two analysts and four developers (6 in a total) pointed out non-compliance with the Traceability factor and partial compliance with the Automated Support factor, as shown in Figure 3.

Change control in requirements was done through Microsoft Word and by the SVN tool used for version control. A template of the requirements document was created and used by the whole team.

Two developers pointed out non-compliance with the Change History factor. The remaining of the team evaluated it as partially compliant. A developer reported partial compliance with the Consolidated Information factor. In his opinion (Q13), the RSD artifact structure should provide an hyperlink to access each AC+ more quickly given that they were defined in a separate document.

Finally, we investigated the structure of the RSD artifact compared to other approaches previously used by the software engineers (Q12). Six interviewees pointed out that the RSD artifact structure is more appropriate than the structure of the other approaches. No interviewee indicated that the RSD artifact structure was worse than other approaches. In the comparative evaluation by the quality factor (Q13), the majority of interviewees considered that the RSD artifact structure is better in Simplicity, Acceptance Criteria, Technical Aspects and Consolidated Information factors, and equal in the others factors: Non-Functional Requirements, Automated Support, Traceability, Functional Requirements and Change History.

As the RSD artifact structure did not represent the AC+ priority, if an AC+ had a different priority or it had to be implemented in another sprint, the analysts registered this information as a note in Redmine. After the second month of use of the approach, the team modified the RSD artifact structure in order to highlight the priority of the acceptance criteria in the RSD artifact itself (Q31).

The AC+s were categorized by the system analysts as General (potential to be reused in other requirements) or Specific to a particular requirement. This categorization was not part of the structure initially defined for the RSD artifact. However, analysts adopted this practice to facilitate the search for AC+ in the repository and to improve the reuse (Q22, Q33).

The results of the interviews (Q14) showed that the software engineers' expectations were met regarding the RSD artifact structure, as follows:

"The RSD artifact structure allows a more integrated view of the rules and constraints applied to a particular requirement. In previous projects, user stories were described independently and unrelated; we did not have an integrated view of a particular requirement." (#3CS1)

4.1.3. RQ2: How does the RSD approach affect the team work?

a) Effort required to specify, code and test.

The RSD approach was quickly absorbed by the team which reported no difficulties in their learning (Q6). As this approach introduces new practices to specify requirements in ASD, we formulated the following hypothesis: *The effort required to specify requirements using RSD in ASD is higher than using other approaches, but the effort to implement using RSD is lower than when implementing using other approaches.* However, in the opinion of the interviewees (Q24), the effort required to specify using RSD is not higher than using other approaches. One analyst considered that less effort was required. The other analyst believes that it is the same. Regarding Q25, half of the developers considered that RSD required less effort to code than other approaches. The other half considered that it requires the same amount of effort. According to most of the software engineers (7), developing from the RSD artifact requires a reasonable effort; the other software engineers (3) consider that developing from the RSD artifact requires little effort (Q23).

The interviews results showed that the approach met the expectations of the developers regarding their productivity and proved to produce a very simple and objective SRS, suitable for coding activities, as follows (Q27):

"The detailing of the requirements through the AC+ makes the SRS more straightforward and objective for the developer." (#1CS1)

According the analysts, the traceability matrix was not useful in the project due to the high effort required to keep it up-to-date (Q21). The TM presents the relationship between requirements and AC+s, so it should be used to aid the impact analysis of change requests in the requirements. The matrix is not an artifact defined by the RSD approach. It was adopted by the company to manage the requirements. In parallel to drawing up the RSD artifact, the TM had to be updated manually. The same happened when the team needed to change an AC+ and/or a requirement. It was identified that the traceability matrix was often out of date. Only ten requirements were updated in the TM. Some tools available in the market, such as Caliber, generate the traceability of requirements automatically. However, the project did not use any of these tools. Instead, the TM was operationalized by using a spreadsheet shared among the team members. As noted by one of the system analysts:

"The traceability matrix is not being properly used in the project because the manual update requires much effort. I do not think it's worth wasting time on this. The matrix should be extracted automatically from the SRS." (#4CS1)

The acceptance tests were conducted by the internal product owner using the RSD artifacts as a reference. This

was pointed out as a good practice (Q22). The tester validated whether the software complies with each AC+. However, the analyst had to register manually each NC found in Redmine, and forward it to the person responsible for the fix. Sometimes, the analyst did not register the related AC+. As a result, the developer lost a long time analyzing the problem and clarifying doubts.

Analyzing the difficulties reported by the team during the acceptance tests and from the interview results (Q15), we argue that the test can be optimized by using a tool to generate a roadmap (checklist) automatically from an AC+. The tester will need only to check, or uncheck, the checklist entries, according to the test result. In addition to registering the NC by associating the related AC+, the tool can send an email notification to the person responsible for making the necessary corrections. The software adoption will increase the productivity of the team. Furthermore, the use of the checklist minimizes the risk of the tester forgetting to validate an AC+, thus contributing to improve the testing quality.

b) NC in the software due to issues in the SRS.

The acceptance tests were conducted weekly to verify the compliance of the software with the customer's needs. At the beginning of the project, the stakeholders did not have a complete understanding of all customer needs given that the software development was done iteratively and incrementally, following the agile principles. The customer's needs were obtained and detailed through partial validations of the software in each sprint. From the analysis of the Redmine records, six types of NC in the software were identified as shown in Table 5.

Table 5: Types of Non-Conformities - case study 1

	ID	Type of Non-Conformity (NC)
Bugs	#1	Problems of legibility, clarity or completeness in the RSD artifact
Bugs	#2	Customer's needs not represented correctly in the RSD artifact
Bugs	#3	Requirement not implemented according to the RSD artifact
Improvements	#4	New requirements not previously captured but essential for the customer
Improvements	#5	Additional information in requirements already validated by the customer
Changes	#6	Changes in requirements already validated by the customer

During the case study period, 157 NCs were found. 20% (31) of these NCs+ were due to issues in the RSD artifact (Table 5, items #1 and #2) which required corrections in it. The other NCs (126) were not related to the RSD artifact failures. They referred to bugs arising from the implementation of the requirement differently from what was defined in the RSD artifact (#3), improvements (items #4 and #5), and changes in functionality previously requested and already validated by the customer (#6).

It was not possible to assess whether the use of the

RSD approach reduced the NCs resulting from problems in the SRS, because the acceptance tests were conducted in a different way in the previous projects. Another document (test cases) was used to guide the tests before the adoption of the RSD approach. Moreover, the test results were grouped by tested functionality, and a single issue was recorded in Redmine describing all NCs found. After the introduction of the approach, the company decided to conduct the tests from the ACs+ and to register the test results grouping them according to the ACs+, instead of grouping them according to the functionalities.

c) Impact analysis of change requests.

The requests for improvements or changes in requirements were analyzed by the product owner, the analyst, and the developer responsible for the coding. They discussed the impact on the project schedule. The traceability matrix was not used by them to support the impact analysis of change requests because it was outdated most of the time (Q29).

Depending on the impact of the change request, new prioritizations and distributions of requirements were made for the next sprints. However, the initial duration of the sprints was always maintained. It was not modified as a result of the requests. During the months of the case study, the project received a total of 54 change requests - it means that 69% of the RSD artifacts (27) received change requests. These changes were requested during acceptance tests or at weekly customer meetings.

The changes in the artifacts were not perceived as a negative factor. According to the team (Q28, Q30), despite the large number of change requests, the impact was reduced given that the software validations were frequent and the project architecture was scalable. The team pointed out that the conceptual modeling was a positive factor that contributed to the construction of a more stable data model. So, little changes were required in the structure of the data model during the sprints, which reduced the impact of the rework in software.

d) Reuse.

The team reported that due to the TM being outdated, they did not use it to identify AC+s that could be reused (Q16). At the end of the case study, the TM was updated to examine the reuse rate of the AC+s. Many AC+s were reused in several requirements, especially the AC+ of validation, technical, and interface types, resulting in a reuse rate greater than 60% which improved the productivity, as described below:

"The RSD approach has contributed to the reuse of AC+s used in other requirements, improving our productivity." (#6CS1)

By analyzing the TM, it was possible to identify that reuse depended largely on the knowledge about the existence of the AC+s. There were high reuse rates of the AC+s specified by the same analyst. However, the reuse rate of the AC+ specified by different analysts was much

lower. Analysts have reported difficulties to find opportunities for reuse because AC+s were stored as a text document (Q21). A more efficient mechanism is required to share and find AC+s. The support of a tool can increase reuse rates. Besides storing the AC+s in a database, a tool could provide features to locate them more efficiently. Despite the difficulties previously presented, the team considers that the reuse of AC+s improved their performance (Q22).

e) Knowledge transfer, Customer collaboration and Dependence between stakeholders.

During the months of the case study, the trainees were replaced twice. The RSD artifacts were used to reduce the effort to transfer knowledge. So, the rest of the team did not have to interrupt their activities frequently to impart knowledge to new members. Although the team held daily meetings, the RSD artifacts were the main source for developers to code the requirements.

The customer lived in a different city from the development team. Despite this, the customer collaboration was adequate in the team opinion (Q18). Communication with the customer was made on demand through the analysts or PO whenever it was necessary to detail or clarify any requirements. The analysts and developers had frequent interactions. In the opinion of the team (Q19), the frequency and type of communication used by stakeholders were very adequate.

4.2. Case Study 2

4.2.1. Context

The company has adopted Scrum since 2013 using practices such as product backlog, planning poker, frequent prioritization of requirements, retrospective, iterations and daily meetings. Also, XP practices were used such as integration continuous, refactoring and frequent delivery.

The development team consisted of four members: two software engineers, with more than three years of professional experience, and two trainees (university students). In addition, a internal product owner was responsible for the acceptance tests. All members worked in the same activities, there were no different roles. They were responsible for the evolution and maintenance of three applications, one of which is the company's main product that is in operation in more than forty customers.

Before the company adopts the RSD approach, the requirements were specified through stories. In sprint planning, if a story had an estimate of effort greater than 13 points, it was necessary to divide it into two or more stories. Each user story was registered as a ticket in Redmine. The dependency relationships between the user stories were not registered although the Redmine have a resource for this purpose.

From October 2016, the RSD approach has been introduced in one of three development teams to replace the user stories. The approach was used to specify new requirements and change the existing requirements in the

three applications. However, the user stories have been preserved for the team to consult them when necessary.

The team continued using the same tools that used before the introduction of the approach. The Redmine tool was used to control the product backlog and elaborate the RSD artifact. The sprints had monthly duration. The sprint planning considered the product backlog of the three applications, since the team was the same. Most of the AC+ were specified by the team during the sprint planning. Sometimes, other AC+ were specified later by the developers.

4.2.2. RQ1: How does the team evaluate the SRS produced using the RSD approach?

Similar to case study 1, the RSD approach was evaluated according to content and structure.

a) Content.

During the three months of the case study, the team worked in 68 requirements. Most of them (51) already existed previously and the team worked in their evolution. The remainder (17) represented new requirements included in the PB of the three applications after the adoption of the RSD approach.

The conceptual modeling was necessary in 26 requirements, while the modeling of mockups was necessary in 9 requirements. In the other requirements, no mockup was associated with the corresponding RSD artifact because the team used the user interface generated by the ScriptCase tool². 122 AC+s were specified by the team during the investigation period and most of them described new needs of stakeholders (96). The remainder represented changes in functionalities previously described through user stories (26). Sometimes, during the coding activity, the software engineer identified new System-type AC+s and registered them in the corresponding RSD artifact. This occurred in 26 RSD artifacts.

The user stories previously used by the team were written in a very objective and clear way directed to the development team. According to the software engineers, these characteristics remained present in the requirements described using the RSD approach. Software engineers perceived that the RSD approach had improved the completeness and readability of the SRSs, which became more complete by the description of the AC+s with technical aspects, as follows:

"Technical Aspects were not documented before, the corresponding information was only in the minds of the most experienced software engineers." (#1CS2)

According to the team, the grouping of AC+s by functionality provided a consolidated view of a requirement facilitating a general understanding of it (Q14). The readability of the RSD artifacts was considered better than that of the user stories.

²<https://www.scriptcase.com.br/>

Analyzing the changes made in the RSD artifacts (volatility), we observed that only three RSD artifacts were modified during the period of investigation. Regarding the Outdated factor, the team considered that the RSD artifacts were up to date.

b) Structure.

Each RSD artifact was represented in Redmine describing a requirement as a ticket and the AC+s as sub-tickets. Mockups were attached to the ticket as a file, if necessary. The association between the widgets, data entities and AC+ was made in the description field of the ticket. The team relaxed this association and it was only made when there were new widgets, data entities or some related AC+, rather than describing all widgets and data entities, as defined by the approach.

The Redmine was customized to identify the AC+ types. However, the team unified the Validation, Interface and Technical types into a single type called System to facilitate the adoption of the approach in this case study (Q31). AC+ of the System type decreased the dependency among the members helping to meet some expectations of the trainees (Q14). To facilitate the planning and coding activities (Q31), the team extended the approach by creating a new field (Order) in Redmine to identify the order that the AC+ should be coded during the sprint.

The Technical Aspects and Acceptance Criteria factors were evaluated as Very Adequate (Q10). In the opinion of the team regarding these factors (Q13), the structure of the RSD approach has improved over the previously used approach. Despite the Simplicity, Functional Requirements and Consolidated Information factors have been evaluated as in compliance (Q11), the team considered that there was no improvement over the previously used practice (Q13).

Non-functional requirements such as security, accessibility, and storage constraints were defined only at the beginning of the projects. The team reported not being able to evaluate the Non-Functional Requirements factor because no requirements of this type was specified during the investigation period.

The change control in the RSD artifacts was done through Redmine. The Change History factor was evaluated as partial compliant (Q11), as follows:

"The Redmine query to view changes made in the RSD artifacts only had the option to filter by date. It was not possible to search the AC+ that had only the description field modified, for example." (#2CS2)

The structure of the approach was considered non-compliant in the Traceability factor because the relationship between the RSD artifacts had to be done manually. Also, Redmine did not provide any report showing the relationships between RSD artifacts, neither relationships between requirements and AC+. Table 6 summarizes the approach evaluation according to the quality factors and compared to the previously used approach.

Table 6: Evaluation by quality factor - case study 2

	Compliance	Non-Compliance	Partial Compliance	Not assessed
Better	Acceptance Criteria, Technical Aspects, Completeness, Readability	-	Change History, Automated Support	-
Equal	Team-oriented, Simplicity, Functional Requirements, Objectivity, Clearness, Updated, Consolidated Information	Traceability	-	-
Worse	-	-	-	-
Not assessed	-	-	-	NFR

4.2.3. RQ2: How does the RSD approach affect the team work?

a) Effort required to specify, code and test.

The grouping of the AC+s by requirement made it easier to find the specification of a feature to be changed (Q20). As a consequence, the team modified the strategy to specify changes in the existing requirements (Q28). Instead of including a new ticket to specify a change as it was previously done, the new strategy dictated that the software engineer must locate the original ticket and make the change in the related AC+ or include a new AC+. In addition, the AC+s were not finalized after the software validation; they remained active throughout the software lifecycle. This new strategy improved the management of requirements.

In the opinion of trainees (Q26), the description of the AC+ with technical aspects (system type) reduced the effort to code the requirements, as follows:

"Now, we did not need to interrupt our activities to obtain missing information in the SRS. Previously, the acceptance tests were conducted in an exploratory and ad-hoc manner, but the adoption of the RSD approach systematized the tests from the AC+, reducing the required effort." (#3CS1)

b) NC in the software due to issues in the SRS.

Although the AC+s have helped in performing the tests, the team pointed out that the testing activity still is not performed properly in the company and needs to be improved (Q32). Often, features are made available to the customer only with internal tests performed by the developer who coded the functionality, without the PO performing the acceptance tests. In these cases, the developer fixed the identified NCs without registering them. Therefore, we couldn't investigate the NCs in the software re-

sulting from problems in the RSD artifacts, because there wasn't any recording of these NCs.

c) Impact analysis of change requests.

The impact analysis of changes was done by the internal product owner and the senior software engineer. As a result of this analysis, the priority of the requirements and the AC+s was rescheduled, if necessary. As previously mentioned, the projects had no information on the traceability of requirements and AC+. However, the internal product owner pointed out that the absence of this information did not affect the analysis because he and the senior engineer had a thorough knowledge of the applications.

d) Reuse.

It was observed that the percentage of reused AC+s was very low, less than 15%. As previously mentioned, the projects were already in progress when the approach was introduced, so the focus was on the evolution of applications. Often, a single AC+ was grouping the description of several constraints instead of describing only one constraint. This made it difficult to reuse AC+.

As stated earlier, the team did not have good visibility on the relationship between the AC+s and the requirements. Perhaps, access to this kind of information could help software engineers to identify and reuse AC+ used in similar requirements.

It is necessary to investigate over a longer period and in new projects to have a more effective evaluation if the RSD approach can increase the reusability of AC+ in the company.

e) Knowledge transfer, Customer collaboration and Dependence between stakeholders.

Although the AC+ of the system type has reduced stakeholder dependence and the effort required to code a requirement, in the opinion of trainees (Q19), daily meetings continued to be essential to clarify doubts related to AC+ of the business type and to share difficulties with other team members. In the opinion of the two most experienced engineers, the RSD artifacts were the main source of information they used to code a feature.

According to the team (Q18), the communication with the customer was adequate. All team members interacted directly with the customer whenever necessary, or at least once a week. The most frequent communication was done via email, telephone, and instant messaging applications. The RSD artifacts were not used to validate requirements with customers. The validations were always done through frequent software deliveries.

5. Discussions

5.1. Cross-case Analysis

Table 2 summarizes the context of the two empirical studies which evaluated the RSD approach. Some characteristics are common to both companies, such as size, type of software developed and sprint duration. However,

there are some differences in the context of the two conducted assessments. In the first case study, the RSD approach was used since the beginning of the project and for a longer period (12 months). On the other hand, the second case study introduced the approach in three projects related to software systems that were already in operation in more than 40 customers and the evaluation was carried out only for three months. Other differences can be highlighted, such as the tools used to specify requirements, the practices used in the RSD approach, the team size, and the roles within the development team.

Despite the differences in the context of the two case studies, we identified some similarities regarding how the requirements specification activities affect the team work. Table 7 presents a summary of the evaluations conducted in the two case studies. Some factors improve the team performance (+) while others undermine the team performance (-).

We can see in Table 7 that regarding the structure of the RSD artifact, in the two case studies, the integrated view of AC+s, mockups and data entities (#1) and the grouping of AC+ by functionality (#2) improved the readability of the RSD artifacts (#5). Another positive factor was the use of AC+ describing the business and system requirements in an integrated manner (#3), which made the SRS more complete (#6).

Although the AC+s contributed to systematize and improve the performance of the acceptance tests (#15), the two companies presented difficulties in registering the test results (#11) without the support of a specialized tool. Another negative factor was the inadequate support to track the AC+ (#12).

The teams did not have difficulties in using the RSD approach. The learning was very fast (#16). The conceptual modeling and the modeling of mockups were practices already utilized in both companies but they were not integrated into the user stories. The innovation for the two companies was the specification of the AC+s and their integration with the mockups and data entities.

In order to facilitate reuse, the team categorized the AC+ into Specific and General. The team from the first case study also created new notations to represent mandatory and read-only widgets. In the second case study, the association between the AC+s, widgets and data entities was relaxed and it was only made when it was necessary to describe some AC+ or if a new data entity or widget was identified. This flexibility in adapting the approach to the development process used by the company (#17) was pointed as a positive factor.

Some characteristics pointed out in Table 7 were only identified in one of the case studies. For example, the description of the requirements in a clear (#7) and objective (#8) manner were highlighted in the first case study as factors that have contributed to improve the developer's productivity. Although these factors were also present in the RSD artifacts of the second case study, the team pointed out that they were already present in the user stories pre-

Table 7: Summary of the evaluations

	id	Factors	CS 1	CS 2
Structure	#1	Integration of the AC+s, Mockups and Concepts	+	+
Structure	#2	Grouping AC+s by functionality	+	+
Structure	#3	AC+s describing system and business requirements	+	+
Structure	#4	Lack of representation of the AC+ Priority	-	
Content	#5	Readability	+	+
Content	#6	Completeness	+	+
Content	#7	Clearness	+	
Content	#8	Objectivity	+	
Content	#9	A RSD artifact describing multiple requirements	-	
Content	#10	AC+ grouping several constraints		-
Others	#11	Inadequate support to register the acceptance tests by AC+	-	-
Others	#12	Inadequate support to track the AC+	-	-
Others	#13	Reuse of AC+	+	
Others	#14	The mechanism to search the AC+ in the repository	-	
Others	#15	Acceptance Tests from AC+	+	+
Others	#16	Learning curve to use the RSD approach	+	+
Others	#17	Adapting the RSD approach to the company process	+	+

viously used. This means that those factors didn't affect the team's productivity in the second case study. In this case, the corresponding cell is empty.

In the first case study, the team extended the approach to represent the priority of AC+ that was not represented in the initial structure of the approach (#4). Legibility was compromised when several requirements were described in the same RSD artifact (#9), which undermined the productivity of the developers. The specification tool (Redmine) used in the second case study avoided this problem because each requirement was described as an independent ticket. On the other hand, in the second case study, often a single AC+ was grouping the description of several constraints (#10), instead of describing only one constraint as specified in the first case study.

The reuse of AC+s (#13) was pointed out as a factor that contributed to improve the productivity of the team in the first case study. However, the reuse depended largely on knowledge about the existence of AC+s. There were high reuse rates of the AC+s specified by the same analyst. The reuse rate of the AC+ specified by different analysts was much lower because they had difficulties to find AC+s in the repository operationalized by a text document (#14). A more efficient mechanism was required to share and find AC+s. This problem was not reported in the second case study because the reusability of requirements was very low.

Figure 4 shows a model summarizing all the factors detailed in Table 7, indicating if the factor itself improve or undermine team performance (symbol + or - in the lower right corner of the rectangle), and how the factors relate to each other. In this latter case, an arrow from a factor

A with symbol + to a factor B means that factor A contributes to factor B. Similarly, an arrow from a factor A with symbol - to a factor B means that factor A undermines factor B.

Finally, Table 8 summarizes the challenges present in Requirements Engineering for ASD (identified both in literature and through the results from [18]) and how these challenges are addressed by the RSD approach according to the results of the case studies conducted.

Table 8: Summary of the challenges addressed by RSD

Challenges in RE for ASD	How the challenges are addressed by RSD
Inadequate dealing with non-functional requirements [6, 11, 18].	In case study 1 (one), 90% of the respondents pointed out that the RSD approach adequately addresses Non-Functional Requirements through N-type (Non-Functional) acceptance criteria (AC+).
User Stories lack of expressiveness to describe design requirements [4, 8, 12, 14, 16, 18].	The content of the SRS was evaluated in both case studies and their results pointed out that the RSD approach describes design requirements through the systematic use of conceptual modeling, interface prototyping and the acceptance criteria (AC+) of specific types: T (Technical), I (Interface) and N (Non-Functional).
The format to specify requirements [8, 12, 18].	The structure of the RSD artifact was evaluated in both case studies. The results pointed out that integrating mockups, conceptual modeling, and acceptance criteria (AC+) into a single artifact increased development team productivity when the interviewees compared it to previous projects using other requirements specification techniques.
Inadequate prioritization of the requirements [8, 11, 12].	In case study 1 (one), the acceptance criteria (AC+) were not prioritized, which compromised the productivity of the development team. In the second case study, this limitation didn't occur because the structure of the RSD artifact was changed to incorporate the prioritization of AC+.
Lack of automated support to the requirements activity [9, 12, 18, 32].	Both case studies indicated that team productivity was hampered by the lack of a tool to support the acceptance criteria (AC+) traceability and to automatically register acceptance tests from the AC+. According to a previous study performed in 6 (six) companies [18], automated support is an organizational quality factor that does not depend on the technique used to specify requirements. That is, the same problem occurs when there are no tools to support other techniques, such as User Stories.

5.2. Limitations and Threats to Validity

The subjectivity about how the RSD approach affects the team work on the team's perception was a threat. The quality factors defined in the model [18] were used to minimize the impact of this subjectivity.

The conduction of the evaluation by the authors may have produced some bias. This problem could be reduced if other researchers or another research method could be used. Although, certainly, the level of detail captured might be different if the evaluation was carried out by others, since the objective of the authors conducting the case study was to identify *in loco* the limitations and the difficulties that the team had in using the approach and

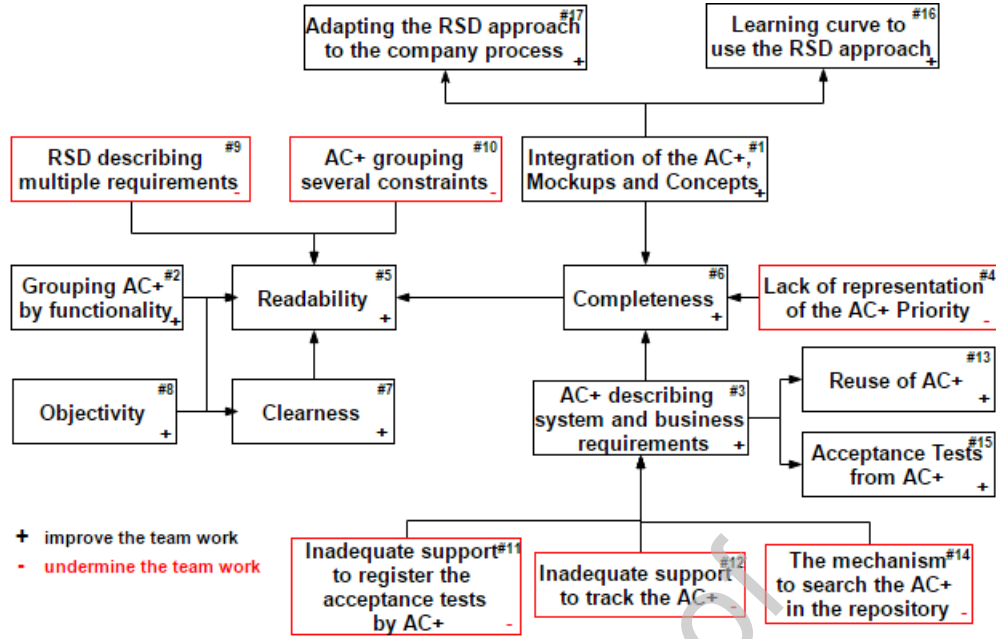


Figure 4: Factors that affect the team work using the RSD approach

also to identify best practices and strengths. In the following, we address the validity of our results from the three perspectives proposed by Merriam [26]:

5.2.1. Internal validity

Regarding the internal validity, to ensure that the results represent the reality, we interviewed team members with different profiles in each study. A threat was the possibility that the software engineers have answered what the researchers wanted to hear, rather than giving their real opinion. To minimize this threat, the interviewees were encouraged to criticize the approach and to point out the difficulties and limitations faced by them when using it, as this information would be used to improve the approach. Besides, the researchers had no personal or professional relationship with the software engineers, and it was reinforced, during the interviews, the importance to detail the answers as much as possible. Also, leading questions were avoided, and probes were defined with the objective of deepening the respondent's answers, avoiding direct, vague and superficial responses. Thus, we tried to get arguments, examples, and details to help understanding the perception of each software engineer.

To mitigate the threat to validity related to data extraction and to increase credibility, data were triangulated through interviews, observations, and analysis of documents. Also, we used member checking [26], also known as respondent validation, to avoid misinterpretations of what participants said.

5.2.2. Reliability

An important question in qualitative research is whether the findings are consistent with the data collected [26].

Reliability is related to the extent to which the research findings can be replicated by the same or other researchers. The question is not whether the findings will be found again but whether the results are consistent with the data collected.

We do not expect that the results of each study will be reproduced. Even if we repeat the same protocol, in the same companies, with a similar set of participants, their perception of the RSD Approach and the organizational context are likely to evolve. To increase the reliability, we used data triangulation and peer review of the findings with another researcher. Also, we described in detail the methods and procedures to conduct the studies.

5.2.3. External validity

External validity is concerned with the extent to which the research findings can be applied or used in contexts different from those in which the study was first conducted [26]. This is related to how generalizable the results are.

The strategy used to promote the transferability of the results was to apply the research instruments in two studies. First, we provided rich descriptions of each company investigated to contextualize the study, so that readers will be able to determine the extent to which their situations match to the research context, and, hence, whether the findings can be transferred. Then, we investigated the differences and similarities between the two contexts to build an integrated view of the results from both studies. In addition, the sample of participants intended to achieve a maximum variation to provide richer data.

6. Related Work

In this section, we discuss some related works. These are divided into two categories: (i) Approaches to specify requirements in ASD, and (ii) Empirical Studies to evaluate requirements specification techniques used in ASD.

6.1. Approaches to specify requirements in ASD

Six proposals of new approaches to specify requirements in ASD were identified, namely Nawrocki et al. [15], Losada et al. [33], Batool et al. [32], Rivero et al. [34], Gebhart et al. [35] and Wanderley et al. [36]. These studies were analyzed according to some factors of the quality model defined in [18] and briefly explained in section 2. Each factor was evaluated as: i) Addressed (+): if the factor was fully addressed; ii) Partially Addressed (0): if the factor was addressed partially; iii) Not Addressed (-): if the factor was not addressed. Also, these proposals are compared according to the type of technique used to specify requirements. Table 9 summarizes the results of this analysis.

An extension of the XP (eXtreme Programming) is proposed by Nawrocki et al. [15] by introducing new requirements engineering practices. The authors assert that an SRS based on User Stories is not enough for the developer. Based on some practices of CMMI [37] and some quality attributes of ISO 830 [38], they propose to link tests cases to requirements and include the use of scenarios. The extension suggests the adoption of user interface prototypes and the use of practices to identify NFR. SRSs are used for requirements validation with the customer. Conceptual modeling and technical aspects are not addressed by the extension.

Losada et al. [33] proposed a methodology called InterMod that combines agile practices, Model-Driven Development (MDD), and user-centered design. The authors assert that use cases are insufficient for specifying the requirements, since say little or nothing about how tasks are presented to the user or how they should be positioned in the interface. The methodology is guided by User Objectives that correspond to user desires. A User Objective can fit with one or more functional requirements (use cases) and/or non-functional requirements. So, InterMod suggests the use of six models in order to improve the SRS in agile projects: User, System, Tasks, Prototypes & Behavior, Presentation, and Functionality. The preparation of these models requires a considerable effort. Acceptance criteria are not included in the methodology.

Batool et al. [32] proposed a Scrum framework to improve the RE process. The framework is based on User Stories to describe functional requirements. Class diagrams and user interface prototypes are used in order to facilitate the understanding of the developers, but the SRS lacks non-functional requirements and technical aspects. The framework also adopts other artifacts such as story cards, index cards, and vision document.

Rivero et al. [34] proposed a hybrid Agile, MDD and coding approach called ELECTRA (Extensible modelDriven Enduser CentRiC API) that uses an end-user friendly language to define API (Application Programming Interface)-related requirements based on mockups [34]. The ELECTRA approach is an adaptation of Scrum, and it mandatorily requires building mockups with essential end-user participation to specify the stories. After all User Stories are associated to mockups, developers use a tool to tag the mockups with API-related annotations. Constraint annotations enable the definition of business rules and action annotations, which describe the execution of heterogeneous or complex tasks within the API. Some annotations provide an end-user friendly structured text mode to specify API requirements in natural language. Technical specifications (understood by engineers) are also described. The approach does not use conceptual modeling, neither defines acceptance criteria.

Gebhart et al. [35] argued that scenarios are an appropriate way to describe a system from the user's point of view in ASD, and presents an enhancement of existing scenario-based requirements engineering techniques to achieve the quality characteristics of the international standard ISO 29148 [39]. The methodology proposed establishes three initial activities for the identification of stakeholders, as well as identification and prioritization of goals. The goals are achieved through scenarios that can be reused. The scenarios are customer-oriented, free of implementation details and do not contain architectural decisions. The methodology also requires the construction of other artifacts, such as Glossary, Constraints, Scenarios for non-functional requirements, and Derivation of acceptance criteria from the scenarios. A lot of artifacts ends up compromising the SRS's simplicity and the agility of the process.

Wanderley et al. [36] proposed SnapMind, a visual requirements language based on mind maps to represent both User Stories and domain models for agile development. The framework aims to make the requirements modeling process more user-centered. The process is composed of three activities. The first activity is the software specification (containing the definition of a domain model and User Stories as sub-activities). In the second activity, SRSs are validated by end-users. The third activity is conducted by software engineers to verify the consistency between requirements models. The framework does not support specification of non-functional requirements, mockups, acceptance criteria and, therefore, technical constraints.

In the previously described approaches, the SRS is used to requirements validation with the customer, although the ASD suggests the use of frequent releases through software running for this purpose. This strategy entails an SRS directed to the customer, rather than to the developer. Except for the ELECTRA approach, the SRS focus on the description of functional requirements and does not address technical and design constraints properly. RSD

Table 9: Comparison of RSD with related work

SRS Quality Factors	Related Works						RSD
	Nawrocki (2002)	Losada (2012)	Batool (2013)	Rivero (2014)	Gebhart (2014)	Wanderley (2014)	
Simplicity	0	-	0	+	-	+	+
Team-Oriented	0	0	0	0	-	0	+
Acceptance Criteria	+	-	0	-	+	0	+
Non-Functional Requirements	+	+	-	+	0	-	0
Technical Aspects	-	0	-	+	0	0	+
Functional Requirements	+	+	+	+	+	+	+
Consolidated information	0	-	0	0	0	0	0

Key: + Addressed 0 Partially Addressed - Not Addressed

SRS driven by:							
Goals / Objectives		X			X		
User Stories	X		X	X		X	
Scenarios / Use Cases	X	X			X		
Tasks / Activities		X					
Mind Maps						X	
Mockups	X	X	X	X			X
Conceptual Model			X			X	X
Acceptance Criteria							X

approach, on the other hand, innovates by systematizing the use of acceptance criteria, mockups and conceptual modeling in ASD, and integrates the description of the functional and technical requirements in a single view in order to provide an SRS with the information required for coding.

6.2. Empirical Studies to evaluate requirements specification techniques used in ASD

Two empirical studies to evaluate techniques to specify requirements in ASD were identified, namely the works of Lucassen et al. published in 2016 [40] and in 2017 [41].

The first empirical study applied a survey to software practitioners and performed semi-structured interviews with a sample of these practitioners to evaluate the effectiveness of User Stories [40]. The survey collected 182 responses and 21 practitioners were interviewed. The work resulted in 12 findings regarding the use of User Stories to specify requirements in ASD. These findings indicate, for example, that practitioners who use quality guidelines and templates to create User Stories are significantly more positive regarding the impact of User Stories on productivity and work deliverable quality. Also, technical stakeholders are less positive regarding the effectiveness of templates and quality guidelines than non-technical stakeholders. This latter evidence corroborates with the findings obtained from the empirical study reported in [18] which motivated the creation of RSD.

The second empirical study investigated the impact of introducing a method and a tool proposed in [13] to reduce syntactic defects that violate desired qualities of requirements specified using User Stories [41]. The purpose was

to evaluate the impact of the Quality User Story (QUS) framework and the computational linguistics tool Automatic Quality User Story Artisan (AQUSA) on practitioners' work. QUS and AQUSA were used by 30 practitioners from 3 companies over a period of 2 months. This multiple case study found that the treatment (applying QUS and AQUSA) didn't affect the practitioners perception on User Stories quality, although it has, in fact, increased. Also, there is no significantly improvement on the productivity and work deliverable quality. On the other hand, practitioners' perception indicates that communication frequency and effectiveness improved. The authors concluded that it is necessary to perform more empirical studies to find more conclusive results regarding the use of QUS and AQUSA in the industrial practice. The evaluation of RSD, presented in this paper, found evidences indicating that the RSD approach can improve the team performance. Also, practitioners' perception indicated that the completeness and readability of the SRSs have improved by using the RSD approach when compared to previously used techniques to specify requirements in ASD (including User Stories).

7. Conclusions

The goal of the case studies was to evaluate the use of the RSD approach in practice and to identify its strengths and limitations. The design and procedures employed to conduct these empirical studies were described in detail and they can be used by other researchers in other contexts to develop new studies.

Despite the differences in the context of the two case studies, the findings present many similarities regarding how the research questions were answered. For RQ1, the results showed that the RSD approach met the developer's expectations and proved to be very effective in producing a more objective SRS, suitable for coding activities. For RQ2, the feedback collected through interviews suggests that the RSD approach does not add extra effort for specification and may also reduce the effort for coding, testing and maintaining software. The practices introduced did not adversely affect the process agility. The performance of the tests created from the AC+ was also pointed out as a positive factor. Thus, specifying requirements using RSD affects positively the team work.

Regarding the challenges using the RSD approach, the productivity of the teams was compromised when several requirements were specified in the same RSD artifact or when an AC+ was grouping several constraints. The mechanism to search AC+s in the repository chosen by the team is inefficient, which made reusability difficult. The teams also pointed out the inadequate support to track the AC+s as well as to register the acceptance tests created from the AC+s. These and other improvements, such as prioritization and categorization of the AC+, and the need of a tool to support the elaboration of the RSD artifact, are required for the RSD approach. More evaluations of the RSD approach are still required to assess, for example, its impact on development iterations and on architecture stability, the amount of rework required when moving from one iteration to another, and how it works in different contexts, such as mobile applications and projects with distributed teams. Also, in a future case study, we intend to use the same evaluation instruments as Lucassen et al. [40] to evaluate the effectiveness of using RSD in practice and to compare its results with the results of the evaluation of User Stories effectiveness reported in [40].

References

- [1] Version One, The 10th Annual State of Agile Report, Tech. rep., Version One (2017 - accessed on January 8, 2018). URL <https://versionone.com/pdf/VersionOne-10th-Annual-State-of-Agile-Report.pdf>
- [2] R. Kasauli, G. Liebel, E. Knauss, S. Gopakumar, B. Kanagwa, Requirements engineering challenges in large-scale agile system development, in: 25th IEEE International Requirements Engineering Conference (RE), 2017, pp. 352–361. doi:10.1109/RE.2017.60.
- [3] M. Daneva, E. Van Der Veen, C. Amrit, S. Ghaisas, K. Sikkil, R. Kumar, N. Ajmeri, U. Ramteerthkar, R. Wieringa, Agile requirements prioritization in large-scale outsourced system projects: An empirical study, *Journal of Systems and Software* 86 (5) (2013) 1333–1353. doi:10.1016/j.jss.2012.12.046.
- [4] A. Read, R. Briggs, The many lives of an agile story: Design processes, design products, and understandings in a large-scale agile development project, in: 45th Hawaii International Conference on System Science (HICSS), IEEE, 2012, pp. 5319–5328. doi:10.1109/HICSS.2012.684.
- [5] S. Wagner, D. Fernández, M. Felderer, M. Kalinowski, Requirements engineering practice and problems in agile projects: Results from an international survey, in: Proceedings of the XX Iberoamerican Conference on Software Engineering, Buenos Aires, Argentina., 2017, pp. 389–402.
- [6] I. Inayat, S. S. Salim, S. Marczak, M. Daneva, S. Shamshirband, A systematic literature review on agile requirements engineering practices and challenges, *Computers in Human Behavior* 51 (2015) 915 – 929, Computing for Human Learning, Behaviour and Collaboration in the Social and Mobile Networks Era. doi:10.1016/j.chb.2014.10.046.
- [7] E.-M. Schön, D. Winter, M. J. Escalona, J. Thomaschewski, Key challenges in agile requirements engineering, in: International Conference on Agile Software Development, Springer, 2017, pp. 37–51. doi:10.1007/978-3-319-57633-6_3.
- [8] V. T. Heikkilä, D. Damian, C. Lassenius, M. Paasivaara, A mapping study on requirements engineering in agile software development, in: 41st Euromicro Conference on Software Engineering and Advanced Applications (SEAA), IEEE, 2015, pp. 199–207. doi:10.1109/SEAA.2015.70.
- [9] W. Alsaqaf, M. Daneva, R. Wieringa, Quality requirements in large-scale distributed agile projects – a systematic literature review, in: P. Grünbacher, A. Perini (Eds.), Requirements Engineering: Foundation for Software Quality, Springer International Publishing, Cham, 2017, pp. 219–234. doi:10.1007/978-3-319-54045-0_17.
- [10] K. Curcio, T. Navarro, A. Malucelli, S. Reinehr, Requirements engineering: A systematic mapping study in agile software development, *Journal of Systems and Software* 139 (2018) 32 – 50. doi:10.1016/j.jss.2018.01.036.
- [11] B. Haugset, T. Stalhane, Automated acceptance testing as an agile requirements engineering practice, in: 45th Hawaii International Conference on System Sciences, 2012, pp. 5289–5298. doi:10.1109/HICSS.2012.127.
- [12] P. Heck, A. Zaidman, A systematic literature review on quality criteria for agile requirements specifications, *Software Quality Journal* 26 (1) (2018) 127–160. doi:10.1007/s11219-016-9336-4.
- [13] G. Lucassen, F. Dalpiaz, J. van der Werf, S. Brinkkemper, Improving agile requirements: the quality user story framework and tool, *Requirements Engineering* 21 (3) (2016) 383–403. doi:10.1007/s00766-016-0250-x.
- [14] N. Abdullah, S. Honiden, H. Sharp, B. Nuseibeh, D. Notkin, Communication patterns of agile requirements engineering, in: Proceedings of the 1st Workshop on Agile Requirements Engineering, AREW '11, ACM, New York, NY, USA, 2011, pp. 1:1–1:4. doi:10.1145/2068783.2068784.
- [15] J. Nawrocki, M. Jasiński, B. Walter, A. Wojciechowski, Extreme programming modified: Embrace requirements engineering practices, in: Proceedings of the 10th Anniversary IEEE Joint International Conference on Requirements Engineering, RE '02, IEEE Computer Society, Washington, DC, USA, 2002, pp. 303–310.
- [16] H. Sharp, H. Robinson, M. Petre, The role of physical artefacts in agile software development: Two complementary perspectives, *Interact. Comput.* 21 (1-2) (2009) 108–116. doi:10.1016/j.intcom.2008.10.006.
- [17] J. Medeiros, D. Alves, A. Vasconcelos, C. Silva, E. Wanderley, Requirements engineering in agile projects: a systematic mapping based in evidences of industry, in: ESELAW, CIBSE - Ibero-American Conference on Software Engineering. Peru., Universidad Católica San Pablo, 2015, pp. 466–479.
- [18] J. Medeiros, A. Vasconcelos, C. Silva, M. Goulão, Quality of software requirements specification in agile projects: A cross-case analysis of six companies, *Journal of Systems and Software* 142 (2018) 171 – 194. doi:10.1016/j.jss.2018.04.064.
- [19] J. Medeiros, A. Vasconcelos, M. Goulão, C. Silva, J. a. Araújo, An approach based on design practices to specify requirements in agile projects, in: Proceedings of the Symposium on Applied Computing, SAC '17, ACM, New York, NY, USA, 2017, pp. 1114–1121. doi:10.1145/3019612.3019753.
- [20] K. Beck, M. Beedle, A. Van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. Martin, S. Mellor,

- K. Schwaber, J. Sutherland, D. Thomas, Manifesto for agile software development (2001 - accessed on January 8, 2018). URL <http://agilemanifesto.org/>
- [21] J. Ferreira, J. Noble, R. Biddle, Agile Development Iterations and UI Design, in: Agile 2007 (AGILE 2007), 2007, pp. 50–58. doi:10.1109/AGILE.2007.8.
- [22] F. Medeiros, J. Medeiros, F. Ayres, C. Viana, J. Rocha, V. Viagas, E. Mendes, A. Santos, An information system to support the anti-doping process, in: K. J. Kim, N. Joukov (Eds.), Information Science and Applications (ICISA), Springer Singapore, Singapore, 2016, pp. 97–107.
- [23] R. Yin, Case Study Research. Design and Methods, 4th Edition, Sage Publications, London, 2009.
- [24] S. Easterbrook, J. Singer, M.-A. Storey, D. Damian, Selecting empirical methods for software engineering research, in: F. Shull, J. Singer, D. I. Sjøberg (Eds.), Guide to advanced empirical software engineering, Springer-Verlag, London, 2008, Ch. 11, pp. 285–311.
- [25] C. Shields, Can case studies achieve the “gold standard”? or when methodology meets politics, in: The Annual Meeting of the American Educational Research Association, 2017.
- [26] S. Merriam, Qualitative Research: A Guide to Design and Implementation, 3rd Edition, Jossey-Bass, 2009.
- [27] P. Runeson, M. Höst, Guidelines for conducting and reporting case study research in software engineering, Empirical software engineering 14 (2) (2009) 131. doi:10.1007/s10664-008-9102-8.
- [28] S. Ashmore, K. Runyan, Organizational culture considerations with agile, in: Introduction to Agile Methods, 1st Edition, Addison-Wesley Professional, 2014, Ch. 2, pp. 17–21.
- [29] C. B. Seaman, Qualitative Methods, in: F. Shull, J. Singer, D. Sjøberg (Eds.), Guide to Advanced Empirical Software Engineering, Springer London, London, 2008, pp. 35–62. doi:10.1007/978-1-84800-044-5_2.
- [30] N. Northcutt, D. McCoy, Interactive Qualitative Analysis: A Systems Method for Qualitative Research, Sage Publications, Thousand Oaks, CA, 2004.
- [31] Agile Alliance, Agile Glossary (2016 - accessed on March 7, 2018). URL <https://www.agilealliance.org/agile101/guide-to-agile/agile-glossary/>
- [32] A. Batool, Y. Motla, B. Hamid, S. Asghar, M. Riaz, M. Mukhtar, M. Ahmed, Comparative study of traditional requirement engineering and agile requirement engineering, in: 15th International Conference Advanced Communication Technology (ICACT), 2013, pp. 1006–1014.
- [33] B. Losada, M. Urretavizcaya, I. Fernández-Castro, A Guide to Agile Development of Interactive Software with a "User Objectives"-driven Methodology, Sci. Comput. Program. 78 (11) (2013) 2268–2281. doi:10.1016/j.scico.2012.07.022.
- [34] J. Rivero, J. Grigera, G. Rossi, E. Luna, F. Montero, M. Gaedke, Mockup-driven development: Providing agile support for model-driven web engineering, Information and Software Technology 56 (6) (2014) 670 – 687. doi:10.1016/j.infsof.2014.01.011.
- [35] M. Gebhart, P. Giessler, P. Burkhardt, S. Abeck, Quality-oriented requirements engineering for agile development of restful participation service, in: ICSEA: The Ninth International Conference on Software Engineering Advances, 2014, pp. 69–74.
- [36] F. Wanderley, A. Silva, J. Araujo, D. S. Silveira, Snapmind: A framework to support consistency and validation of model-based requirements in agile development, in: 4th International Model-Driven Requirements Engineering Workshop (MoDRE), IEEE Computer Society, Los Alamitos, CA, USA, 2014, pp. 47–56. doi:10.1109/MoDRE.2014.6890825.
- [37] SEI, CMMI for Development. Version 1.3 (CMU/SEI-2010-TR-033), Tech. rep., Software Engineering Institute (2010). URL <http://resources.sei.cmu.edu/library/asset-view.cfm?AssetID=9661>
- [38] IEEE, 830-1998 – IEEE Recommended Practice for Software Requirements Specifications, Standard 830-1998, IEEE (Oct. 1998). doi:10.1109/IEEESTD.1998.88286.
- [39] ISO/IEC/IEEE, 29148-2011 – ISO/IEC/IEEE International Standard - Systems and software engineering – Life cycle processes –Requirements engineering, Standard 29148-2011, ISO/IEC/IEEE (Dec. 2011). doi:10.1109/IEEESTD.2011.6146379.
- [40] G. Lucassen, F. Dalpiaz, J. van der Werf, S. Brinkkemper, The use and effectiveness of user stories in practice, in: M. Daneva, O. Pastor (Eds.), Requirements Engineering: Foundation for Software Quality, Springer International Publishing, Cham, 2016, pp. 205–222. doi:10.1007/978-3-319-30282-9_14.
- [41] G. Lucassen, F. Dalpiaz, J. van der Werf, S. Brinkkemper, Improving User Story Practice with the Grimm Method: A Multiple Case Study in the Software Industry, in: P. Grünbacher, A. Perini (Eds.), Requirements Engineering: Foundation for Software Quality, Springer International Publishing, Cham, 2017, pp. 235–252. doi:10.1007/978-3-319-54045-0_18.